

Final Project - Topic A: Text Classification Pipeline Forensics

Text Classification Pipeline Forensics

Diagnose a noisy disaster-tweet classifier, reproduce a reference baseline, and explain which pipeline changes are genuinely trustworthy.

I. Background

Text classifiers often look better understood than they really are. A model can improve because a preprocessing decision removed noise, because a threshold matched the evaluation metric, or because the dataset contains shortcuts that will not hold outside the benchmark. A small score change is therefore not a complete answer. You need to know which examples changed, which errors were fixed, which new errors were introduced, and whether the mechanism matches the intended hypothesis.

In this project, you build and investigate a CPU-only text-classification pipeline for the Kaggle Disaster Tweets dataset. The task is binary classification: `target=1` means the tweet describes a real disaster, and `target=0` means it does not. The data source is public; the starter package provides the fixed split and reference contract, but you must download the data and implement the baseline and experiments yourself.

II. Project Overview

The project is organized around five investigation tickets. Each ticket asks you to test one kind of pipeline or data-quality hypothesis and to explain the result with evidence. You should treat the visible held-out score as one artifact in a larger diagnosis, not as the only evidence.

The starter package includes:

- the public source for the full labeled `train.csv` from Disaster Tweets;
- `split_indices.json` with fixed `train_ids`, `dev_ids`, and `heldout_ids`;
- `configs/project_contract.json` with the reference baseline metric and tolerance.

The held-out split is used only after a ticket decision is frozen. Use the dev split for choosing preprocessing settings, thresholds, and model variants. Do not move ids across splits.

III. Starter Package

The starter package provides the fixed split, reference contract, and artifact interface. Use its `README.md` for data-source links, split usage, and output-format expectations. You are responsible for writing the code that downloads, loads, models, evaluates, and audits the data.

IV. Project Content

You must complete the five tickets below. Each ticket should have a hypothesis, one intended lever, a controlled experiment, dev evidence, final held-out evidence, concrete examples, and one limitation.

#	Ticket	Required question
1	Baseline discrepancy diagnosis	Does your TF-IDF + Logistic Regression baseline match the reference? If it does not, which split, seed, version, or preprocessing difference explains the gap? If it does match, which two discrepancy probes show why the reference is reproducible?
2	Text normalization lever	Do URL, mention, hashtag, punctuation, casing, or emoji decisions help or hurt? Which false positives and false negatives move, and does the movement match your hypothesis?
3	Feature and shortcut audit	How much signal comes from keyword, length, or other shallow artifacts? Is that signal legitimate task information, a dataset artifact, or both?
4	Decision-rule and model ticket	Does threshold tuning, class weighting, regularization, or a second CPU classifier improve the operating point? Explain the precision-recall tradeoff, not only the best F1.
5	Data-quality and error ticket	Which duplicates, likely mislabels, ambiguous tweets, or hard negatives limit the score? Separate rows that should be fixed from rows that should be kept, flagged, treated as ambiguous, or rejected as false-positive audit findings.

For Ticket 5, do not edit held-out labels or remove held-out examples. If you test a training-label correction, keep both the original and proposed labels in an audit table.

V. Expected Evidence

Design experiments that support your conclusions rather than merely producing a different score. Your evidence should make it possible to understand what changed, why it changed, and whether the change should be trusted.

You are expected to keep enough artifacts for the results to be regenerated and inspected. The starter README specifies required prediction files, summary tables, and any machine-checkable schemas.

Hidden grading may run stress variants that preserve labels but perturb superficial text signals such as URLs, casing, hashtags, mentions, and metadata-like shortcuts. A strong submission explains whether each signal is legitimate task information, an artifact, or mixed evidence rather than tuning only to the visible held-out split.

VI. Report Requirements

The final report .pdf should summarize your methodology, analysis, key findings, and conclusions. It should be self-contained and written as a coherent project report rather than a collection of command outputs.

A strong report should make clear what problem you studied, what experiments you chose, what evidence supports your conclusions, and where the evidence is uncertain or limited. For this topic, the report should convince the reader that your submitted experiments were motivated by clear hypotheses, evaluated under a controlled setup, and interpreted through concrete prediction changes rather than only score differences.

The report should also contain a dedicated **Difficulties and Solutions** section with at least three concrete, verifiable challenges you encountered during the project and how you addressed them. Include an AI usage declaration explaining how AI tools were used and how their outputs were verified.

Submissions that only list commands and F1 scores cannot receive high credit even if the code runs.

VII. Deliverables

```
repo/
|-- pipeline/
|-- tickets/
|   |-- ticket-1-baseline.md
|   |-- ticket-2-normalization.md
|   |-- ticket-3-shortcuts.md
|   |-- ticket-4-decision-rule.md
|   |-- ticket-5-data-quality.md
|-- experiments/
|-- predictions/
|-- results/
|-- logs/chat.md
|-- report.pdf
`-- README.md
```

The `README.md` must contain the commands needed to regenerate the main results.

VIII. Academic Integrity and AI Usage

Using AI tools is expected. You must cite datasets, notebooks, libraries, and tools. Reproducing the reference baseline is allowed; presenting public code or AI output as unverified original analysis is not.